

DOKUMENTASI PROYEK & PANDUAN CODING

JafariLab

Website Portofolio dan Content Management System

Arsitektur, instalasi, API, database, panel admin, standar pengembangan, deployment, keamanan, dan troubleshooting.

Versi dokumen: 1.0

Tanggal: 29 Juni 2026

Runtime: Node.js + Express + MySQL

Daftar Isi

- | | |
|------------------------------|------------------------------|
| 1. Ringkasan proyek | 12. Frontend publik |
| 2. Teknologi dan dependensi | 13. Pengelolaan upload |
| 3. Arsitektur sistem | 14. Panduan coding |
| 4. Struktur direktori | 15. Pola penambahan fitur |
| 5. Persyaratan dan instalasi | 16. Validasi dan pengujian |
| 6. Konfigurasi environment | 17. Keamanan |
| 7. Menjalankan aplikasi | 18. Deployment dan ngrok |
| 8. Database dan seeding | 19. Backup dan pemeliharaan |
| 9. Alur autentikasi | 20. Troubleshooting |
| 10. Dokumentasi API | 21. Keterbatasan dan roadmap |
| 11. Panel admin dan CMS | 22. Checklist serah terima |

Ruang lingkup: dokumen ini menggambarkan kondisi source code pada saat dibuat. Nilai rahasia dari `.env` tidak disalin ke dokumen.

1. Ringkasan Proyek

JafariLab adalah website portofolio kreatif dengan halaman publik dan panel administrasi terproteksi. Pengunjung dapat melihat profil, layanan, portofolio, statistik, serta informasi kontak. Administrator dapat mengelola proyek, layanan, dan hampir seluruh teks halaman tanpa mengubah source code.

Komponen	Fungsi
Website publik	Menampilkan konten profil, hero, keunggulan, layanan, portofolio, tentang, statistik, kontak, dan footer.
Panel admin	Login, CRUD proyek, CRUD layanan, serta penyuntingan pengaturan website.
REST API	Penghubung browser dengan data MySQL dalam format JSON.
MySQL	Menyimpan administrator, proyek, layanan, dan pengaturan key-value.
Upload lokal	Menyimpan gambar portofolio di <code>public/uploads/</code> .

2. Teknologi dan Dependensi

Teknologi	Peran	Versi proyek
Node.js	Runtime server JavaScript. Proyek memakai ES Modules.	Disarankan Node.js LTS modern
Express	HTTP server, routing, static files, dan middleware.	^4.18.2
MySQL / mysql2	Database dan connection pool berbasis Promise.	mysql2 ^3.22.5
express-session	Session login admin berbasis cookie.	^1.19.0
bcryptjs	Hash dan verifikasi password administrator.	^3.0.3
Multer	Parsing multipart dan upload gambar.	^2.2.0
dotenv	Membaca konfigurasi dari <code>.env</code> .	^16.3.1
HTML/CSS/JavaScript	UI publik dan panel admin tanpa framework frontend.	Native browser

`better-sqlite3` dipertahankan untuk migrasi satu kali dari database SQLite lama ke MySQL. Ia bukan database utama aplikasi saat ini.

3. Arsitektur Sistem



Express melayani file statis dari folder `public`. JavaScript browser mengambil data melalui endpoint `/api/`. Route publik menggunakan query baca, sedangkan route mutasi admin melewati middleware `requireAuth`. Gambar diunggah ke disk lokal dan path-nya disimpan dalam tabel `projects`.

Alur startup

1. `index.js` memuat environment.
2. Middleware JSON, form, session, dan static file dipasang.
3. `initDatabase()` membuat database/tabel bila belum ada.
4. Migrasi SQLite dijalankan bila relevan.
5. Data awal dan akun admin dibuat bila tabel masih kosong.
6. Express mendengarkan pada port dari `PORT`.

4. Struktur Direktori

```
Jafarilab/
├── index.js           # Entry point Express
├── package.json       # Scripts dan dependensi
├── .env               # Konfigurasi lokal (rahasia)
├── config/
│   ├── database.js   # Pool MySQL, migrasi, schema, seed
│   └── portfolio.db*  # Data SQLite lama/migrasi
├── database/
│   └── jafarilab.sql  # Schema MySQL untuk import manual
├── middleware/
│   └── auth.js        # Proteksi endpoint admin
├── routes/
│   ├── auth.js       # Login, status sesi, logout
│   ├── portfolio.js   # CRUD proyek + upload
│   ├── services.js    # CRUD layanan
│   └── settings.js    # Baca/simpan konten website
├── public/
│   ├── index.html     # Halaman publik
│   ├── login.html     # Login admin
│   ├── css/           # Stylesheet publik/admin/responsif
│   ├── js/app.js      # Hidrasi konten publik
│   ├── js/admin.js    # Interaksi CMS
│   └── uploads/       # File gambar proyek
├── views/admin/
│   └── dashboard.html  # Dashboard CMS
└── docs/
    ├── dokumentasi-jafarilab.html
    └── Dokumentasi-JafariLab.pdf
```

5. Persyaratan dan Instalasi

Persyaratan

- Node.js dan npm.
- MySQL Server aktif.
- Akun MySQL dengan izin membuat database dan tabel.
- Windows PowerShell, Terminal, atau shell setara.

Langkah instalasi

```
cd C:\Users\aeeste\JafariLab  
npm install
```

Salin atau buat `.env`, isi koneksi database dan kredensial awal. Pastikan MySQL aktif, lalu:

```
npm start  
# sama dengan:  
node index.js
```

Buka `http://localhost:3000` bila `PORT=3000`.

6. Konfigurasi Environment

Variabel	Wajib	Keterangan	Contoh aman
PORT	Tidak	Port HTTP; fallback source code adalah 5000.	3000
DB_HOST	Disarankan	Host MySQL.	127.0.0.1
DB_PORT	Tidak	Port MySQL; default 3306.	3306
DB_NAME	Tidak	Nama database; default <code>jafarilab</code> .	jafarilab
DB_USER	Ya	Username database.	jafarilab_app
DB_PASSWORD	Ya	Password database.	gunakan password kuat
ADMIN_USERNAME	Seed awal	Dipakai hanya jika tabel admins kosong.	admin
ADMIN_PASSWORD	Seed awal	Di-hash bcrypt ketika akun awal dibuat.	minimal 12 karakter
SESSION_SECRET	Ya	Penandatanganan session; harus acak dan rahasia.	string acak $\geq$ 32 byte

```
PORT=3000
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=jafarilab
DB_USER=jafarilab_app
DB_PASSWORD=ganti_dengan_password_kuat
ADMIN_USERNAME=admin
ADMIN_PASSWORD=ganti_dengan_password_admin_kuat
SESSION_SECRET=ganti_dengan_random_secret_panjang
```

Penting: jangan commit `.env`, password, dump produksi, atau session secret ke repository. Mengubah `ADMIN_PASSWORD` setelah akun dibuat tidak otomatis mengubah password di database.

7. Menjalankan Aplikasi

Perintah	Tujuan
<code>npm start</code>	Menjalankan <code>node index.js</code> .
<code>npm run dev</code>	Mode pengembangan dengan nodemon; nodemon perlu tersedia sebagai dependency/devDependency.
<code>node --check index.js</code>	Memeriksa syntax tanpa menjalankan server.

Menjalankan pada port alternatif

```
# PowerShell
$env:PORT=3100
node index.js
```

Server harus menampilkan `Server berjalan di http://localhost:PORT (MySQL)` .

8. Database dan Seeding

Tabel	Kolom penting	Tujuan
admins	id, username, password_hash, created_at	Akun administrator.
projects	title, category, description, year, image, link, sort_order	Item portofolio.
services	title, description, sort_order	Daftar layanan.
settings	key, value	CMS teks berbentuk key-value.

`config/database.js` membuat database dan seluruh tabel secara idempoten. Seed memakai `INSERT IGNORE` atau pemeriksaan tabel, sehingga restart normal tidak menggandakan data. Jika proyek MySQL masih kosong dan file SQLite lama tersedia, data lama dicoba dipindahkan dalam transaksi.

Import manual

```
mysql -u root -p < database/jafarilab.sql
```

Source code adalah acuan schema runtime yang paling mutakhir. File SQL perlu ikut diperbarui bila schema atau daftar setting berubah.

9. Alur Autentikasi

1. Admin mengirim username/password ke `POST /api/auth/login`.
2. Server mencari username dengan prepared statement.
3. `bcryptjs` membandingkan password dengan hash.
4. Jika benar, `adminId` dan `username` disimpan pada session.
5. Browser menerima cookie session dengan `httpOnly`, `sameSite=lax`, masa aktif 8 jam.
6. Endpoint mutasi memanggil `requireAuth`.
7. Logout menghancurkan session.

`GET /admin` menampilkan login atau dashboard berdasarkan session. `GET /admin/dashboard` mengalihkan pengguna tanpa session kembali ke login.

10. Dokumentasi API

Semua response menggunakan JSON, umumnya berbentuk `{"success": true, "data": ...}`. Endpoint admin memerlukan cookie session valid.

Autentikasi

Method	Endpoint	Auth	Body/hasil
POST	/api/auth/login	Tidak	<code>{username,password}</code> ; membuat session.
GET	/api/auth/me	Ya	Mengembalikan username session atau 401.
POST	/api/auth/logout	Ya	Menghancurkan session.

Portofolio

Method	Endpoint	Auth	Keterangan
GET	/api/portfolio	Tidak	Daftar proyekurut <code>sort_order,id</code> .
POST	/api/admin/portfolio	Ya	Tambah proyek; multipart/form-data.
PUT	/api/admin/portfolio/:id	Ya	Edit proyek; gambar lama dipertahankan bila tidak upload baru.
DELETE	/api/admin/portfolio/:id	Ya	Hapus record proyek.

Field: `title` dan `category` wajib; `description` , `year` , `link` , `sort_order` , serta file `image` opsional.

Layanan

Method	Endpoint	Auth	Keterangan
GET	/api/services	Tidak	Daftar layanan.
POST	/api/admin/services	Ya	Tambah title, description, sort_order.
PUT	/api/admin/services/:id	Ya	Edit layanan.
DELETE	/api/admin/services/:id	Ya	Hapus layanan.

Pengaturan CMS

Method	Endpoint	Auth	Keterangan
GET	/api/settings	Tidak	Seluruh setting sebagai object key-value.
PUT	/api/admin/settings	Ya	Upsert key yang ada pada whitelist server dalam satu transaksi.

Status umum

Status	Makna
200	Operasi berhasil.
400	Input wajib tidak lengkap/tidak valid.
401	Login gagal atau session tidak tersedia.
404	Resource tidak ditemukan.

500

Error server/database; detail dicatat ke console server.

11. Panel Admin dan CMS

Panel admin diakses melalui `/admin`. Navigasi terdiri dari Ringkasan, Portofolio, Layanan, dan Konten Website.

- **Portofolio:** tambah, edit, hapus, upload gambar, atur urutan.
- **Layanan:** tambah, edit, hapus, atur urutan.
- **Konten website:** identitas, topbar, hero, keunggulan, judul section, tentang, statistik, kontak, sosial, dan footer.

`public/js/admin.js` memuat data secara paralel, mengisi form berdasarkan atribut `name`, dan menyimpan melalui API. Penambahan field setting baru membutuhkan sinkronisasi di tiga tempat: field HTML admin, whitelist route settings, dan default seed/database.

12. Frontend Publik

`public/index.html` menyediakan struktur dan fallback content. Setelah halaman dimuat, `public/js/app.js` mengambil settings, proyek, serta layanan secara paralel. Konten kemudian dimasukkan ke DOM. Proyek dan layanan dirender berdasarkan jumlah record sehingga tidak dibatasi tiga item.

Fitur browser meliputi menu responsif, animasi IntersectionObserver, link proyek opsional, link email, dan link media sosial. CSS utama berada di `public/css/style.css`, dengan aturan admin terpisah di `public/css/admin.css`.

13. Pengelolaan Upload

- Jenis file: JPEG, PNG, WebP, dan GIF.
- Batas ukuran: 5 MB.
- Nama file: timestamp + random bytes + ekstensi asli.
- Lokasi: `public/uploads/`.
- Path publik disimpan sebagai `/uploads/nama-file.ext`.

Catatan pemeliharaan: penghapusan proyek atau penggantian gambar belum menghapus file lama dari disk. Buat proses garbage collection berkala agar folder upload tidak terus membesar.

14. Panduan Coding

Prinsip umum

- Gunakan ES Modules: `import / export` , bukan `require` .
- Pisahkan routing, middleware, database, UI, dan aset berdasarkan tanggung jawab.
- Semua query dengan input pengguna wajib memakai placeholder `?` dan prepared statement.
- Endpoint mutasi admin wajib memakai `requireAuth` .
- Validasi input di server; validasi HTML hanya bantuan UX.
- Jangan mengirim stack trace atau konfigurasi rahasia ke client.
- Gunakan transaksi untuk perubahan banyak record yang harus atomik.
- Jaga response API konsisten: `success` , `data` , dan/atau `message` .

Konvensi penamaan

Area	Konvensi	Contoh
JavaScript	camelCase untuk variabel/fungsi	<code>hydrateContent</code>
Database	snake_case	<code>sort_order</code>
Setting keys	snake_case dan prefix section	<code>hero_description</code>
CSS	kebab-case, class semantik	<code>project-visual</code>
Route file	plural sesuai resource	<code>services.js</code>

Pola route yang disarankan

```
router.post('/api/admin/resource', requireAuth, async (req, res, next) => {
  try {
    const value = String(req.body.value || '').trim();
    if (!value) {
      return res.status(400).json({ success: false, message: 'Value wajib diisi.' });
    }
    const [result] = await db.execute(
      'INSERT INTO resources (value) VALUES (?)',
      [value]
    );
    res.json({ success: true, id: result.insertId });
  } catch (error) {
    next(error);
  }
});
```

Aturan frontend

- Gunakan `textContent` untuk teks dari API. Jika harus memakai `innerHTML` , lakukan escaping.
- Jangan membangun URL sensitif langsung dari input tanpa validasi protocol.
- Selalu periksa `response.ok` dan tangani 401 dengan redirect login.
- Jaga progressive enhancement: HTML fallback tetap dapat dibaca ketika API gagal.
- Pastikan elemen baru responsif pada breakpoint yang ada.

Format dan kualitas

Source saat ini cukup ringkas. Untuk pengembangan tim, gunakan ESLint + Prettier, pecah fungsi panjang, dan tambahkan JSDoc hanya pada fungsi dengan kontrak yang tidak jelas. Hindari komentar yang sekadar mengulang kode.

15. Pola Penambahan Fitur

Menambah field CMS

1. Tambahkan input dengan `name="key_baru"` pada dashboard admin.
2. Tambahkan key ke whitelist di `routes/settings.js`.
3. Tambahkan nilai default di `config/database.js`.
4. Hubungkan key ke elemen publik di `public/js/app.js`.
5. Perbarui `database/jafarilab.sql` dan dokumentasi.
6. Uji load, simpan, reload browser, dan restart server.

Menambah resource CRUD

1. Buat/migrasikan tabel.
2. Buat file route dengan GET publik dan mutasi admin.
3. Pasang route di `index.js`.
4. Tambahkan UI admin dan state/rendering.
5. Tambahkan rendering publik jika diperlukan.
6. Uji happy path, input salah, 401, 404, dan error database.

16. Validasi dan Pengujian

Project belum mempunyai automated test; script `npm test` masih placeholder. Minimal verification sebelum rilis:

- `node --check` untuk seluruh file JavaScript server dan browser.
- Server berhasil startup dan terhubung MySQL.
- `GET /`, settings, portfolio, dan services menghasilkan 200.
- Login salah menghasilkan 401; login benar membuat session.
- CRUD proyek dan layanan berfungsi, termasuk upload valid/terlalu besar.
- Pengaturan CMS tersimpan dan tampil setelah reload.
- Logout mengakhiri akses endpoint admin.
- Uji tampilan desktop dan mobile.

Roadmap test

Gunakan Node test runner atau Vitest untuk unit test, Supertest untuk integration API, dan Playwright untuk alur login/CMS. Gunakan database test terpisah dan bersihkan fixture setelah setiap suite.

17. Keamanan

Yang sudah diterapkan

- Password di-hash dengan bcrypt.
- Query input menggunakan parameter.
- Cookie session `httpOnly` dan `sameSite=lax`.
- Mutasi data dilindungi middleware autentikasi.
- Upload dibatasi tipe MIME dan ukuran.
- Setting yang dapat diubah dibatasi whitelist.

Wajib sebelum produksi

- Ganti MemoryStore session dengan MySQL/Redis session store.
- Aktifkan HTTPS dan `cookie.secure=true`; set `app.set('trust proxy', 1)` di belakang proxy.
- Gunakan `SESSION_SECRET`, password admin, dan DB password yang kuat.
- Pasang Helmet, rate limiting pada login, CSRF protection, dan audit log.
- Validasi URL agar hanya menerima `http:` / `https:`.
- Scan konten upload berdasarkan isi file, bukan hanya MIME client.
- Gunakan user MySQL dengan least privilege, bukan root.
- Jangan tampilkan panel admin tanpa kontrol akses jaringan bila tidak diperlukan.

18. Deployment dan ngrok

Preview melalui ngrok

1. Jalankan aplikasi dan pastikan `http://localhost:3000` aktif.
2. Konfigurasi token ngrok sekali: `ngrok config add-authtoken TOKEN`.
3. Jalankan `ngrok http 3000`.
4. Gunakan URL HTTPS yang ditampilkan.

URL ngrok membuka aplikasi lokal ke internet. Gunakan password admin kuat, jangan membagikan token, dan hentikan tunnel setelah selesai.

Deployment produksi

Gunakan process manager seperti PM2 atau service manager OS, reverse proxy Nginx/Apache, HTTPS, database terkelola, session store persisten, dan penyimpanan upload persisten/object storage. Pastikan folder upload tidak hilang saat deploy container baru.

Checklist environment produksi

- Environment rahasia tersimpan di secret manager.
- HTTPS dan domain aktif.
- Backup database dan upload terjadwal.
- Log error dan uptime monitoring aktif.
- Cookie production aman.
- Firewall hanya membuka port yang dibutuhkan.

19. Backup dan Pemeliharaan

Backup MySQL

```
mysqldump -u USER -p --single-transaction jafarilab > jafarilab-backup.sql
```

Restore

```
mysql -u USER -p jafarilab < jafarilab-backup.sql
```

Backup database harus dipasangkan dengan salinan folder `public/uploads`. Uji restore berkala; backup yang belum pernah diuji belum dapat dianggap aman.

Rutinitas

- Perbarui dependency secara terkontrol dan audit kerentanan.
- Rotasi password dan secret.
- Bersihkan file upload yatim.
- Pantau ukuran database, disk, log, error 5xx, dan login gagal.
- Jalankan smoke test setelah setiap perubahan.

20. Troubleshooting

Gejala	Penyebab umum	Solusi
<code>EADDRINUSE :::3000</code>	Port 3000 sudah dipakai proses lain.	Cari PID dengan <code>Get-NetTCPConnection -LocalPort 3000</code> , hentikan proses yang benar, atau gunakan port lain.
Gagal terhubung MySQL	Service mati atau DB_* salah.	Aktifkan MySQL, cek host/port/user/password dan izin user.
Login selalu gagal	Akun seed sudah ada; perubahan .env tidak mengubah record.	Reset password hash melalui prosedur admin/database yang aman.
401 dari endpoint admin	Session kedaluwarsa/tidak terkirim.	Login ulang; cek cookie, hostname, proxy, dan session store.
Upload gagal	Folder tidak ada/tidak writable, MIME salah, atau >5 MB.	Buat folder, perbaiki izin, dan gunakan format yang diterima.
Konten baru tidak tampil	Key belum di-whitelist/default/hydrate.	Ikuti lima titik sinkronisasi pada bagian penambahan field CMS.
Karakter menjadi Â/â	Encoding file/data tidak konsisten.	Simpan source UTF-8, gunakan utf8mb4 MySQL, perbaiki data lama yang mojobake.
<code>npm run dev</code> gagal	nodemon belum terpasang lokal.	<code>npm install --save-dev nodemon</code> atau pakai <code>npm start</code> .

21. Keterbatasan dan Roadmap

1. **Session produksi:** ganti MemoryStore.
2. **Automated tests:** bangun integration dan E2E suite.
3. **Upload lifecycle:** hapus file lama/yatim dan gunakan storage persisten.
4. **Security hardening:** CSRF, Helmet, rate limit, secure cookie, audit log.
5. **Role management:** saat ini hanya satu tipe administrator.
6. **Schema migration:** gunakan migration tool, bukan hanya CREATE TABLE.
7. **Observability:** structured logging, health endpoint, metrics, alerting.
8. **SEO CMS:** title, meta description, OG image, sitemap.
9. **Encoding cleanup:** normalisasi string lama yang menunjukkan mojobake.

22. Checklist Serah Terima

- Source code dan lockfile tersedia.
- `.env.example` tersedia tanpa rahasia.
- Akun admin produksi diserahkan melalui kanal aman.
- Kredensial database menggunakan least privilege.
- Database dan upload sudah dibackup.
- Domain, HTTPS, reverse proxy, dan process manager terdokumentasi.
- Login, CRUD, CMS, email, sosial, dan responsive view diuji.
- Prosedur restore dan rollback diuji.
- Pemilik sistem memahami batasan session/upload saat ini.

Lampiran: Referensi Cepat

```
# Install dan run
npm install
npm start

# Pemeriksaan syntax
node --check index.js
node --check config/database.js
node --check routes/auth.js
node --check routes/portfolio.js
node --check routes/services.js
node --check routes/settings.js
node --check public/js/app.js
node --check public/js/admin.js

# URL lokal
Website: http://localhost:3000/
Admin:   http://localhost:3000/admin

# Tunnel
ngrok http 3000
```